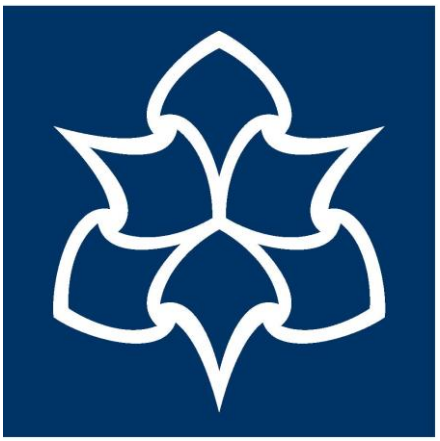
Improving SMS fraud detection by implementing an alert notification system using machine learning.

A REPORT SUBMITTED TO MANCHESTER METROPOLITAN UNIVERSITY FOR
THE DEGREE OF BACHELOR OF SCIENCE
IN THE FACULTY OF SCIENCE AND ENGINEERING



2022

By

Robert Hughes

Department of Computing and Mathematic

Contents

Abstract **vii**

Declaration **viii**

Acknowledgements **ix**

Abbreviations **10**

1 Introduction **11**

1.1 Background 11

1.2 Aims & Objectives 12

1.4 Learning Outcomes 12

1.5 Report Structure 13

2 Literature Review **14**

2.1 Background 14

2.2 Importance of this Issue 14

2.3 Similar Research & Projects 15

2.3.1 Count Vectorizers 16

2.3.2 Naïve Bayes Classifier 16

2.3.3 FastForest Ova ML Model 17

2.4 Conclusion of Related work 18

2.5 Training Material 18

2.6 Evaluation Techniques 19

2.7 Ethical Issues & Concerns 19

3 Product Requirements **20**

3.1 Functionality Requirements 20

3.2.1 Requirements Discovered 20

3.2 Base Layer Requirements 21

4 Design **22**
4.1 Introduction 22
4.2 Approach 22
4.3 User Interface Design 23
4.4 System Architectural Design 24
4.4.1 Machine Learning Model Design 25
4.4.2 System Component Design 26
4.5 Project Timeline Plan 26
4.6 Equality, Diversity & Inclusion 28

5 Implementation **30**
5.1 Introduction 30
5.2 Machine Learning Implementation 30
5.2.1 Dataset Implementation 30
5.2.2 Data Preprocessing 31
5.2.3 ML Model Builder 31
5.3 Software, Algorithms, and Techniques 33
5.4 Code & Challenges 34
5.4.1 User Interface 34
5.4.2 Explanation Generation using Keyword Filtering 37
5.4.3 Error Handling 37
5.4.4 Final Application 38

6 Testing **40**
6.1 Introduction 40
6.2 Testing Methods 40
6.3 Testing Table 41

7 Evaluation **42**
7.1 Testing Evaluation 43
7.2 Comparing to available solutions 44
7.3 Final Evaluation 45

8 Conclusion & Future Work **46**
8.1 Conclusion 46
8.2 My Reflection 46
8.3 Further Project Development Ideas & Suggestions 47

References **48**

List of Appendices

A Creative Piece Showcase Slides 50

B Visual Studio – User Interface Code (Main Page) 54

C Prediction Code (PredictionViewModel.cs Page) 56

D ML Model Consumption (SpamDetector.consumption.cs Page) 58

List of Tables

6.1 Testing Table 36

List of Figures

2.1 Count Vectorizer process. 16

2.2 Sample of the training dataset used 18

4.1 Wireframe designs for the Application 23

4.2 Application Flowchart Diagram 24

4.3 ML Model Selection, Training, and Evaluation. 25

4.4 System Component Design 26

5.1 Spam.csv Dataset. 30

5.2 ML Model Input Code 32

5.3 Final UI Results. 35

5.4 Communication with the model 36

5.5 Explanation generation process 37

5.6 Error Handling, Loading Indicator 38

5.7 Final app version 38

7.1 ML Model Builder Evaluation 43

Abstract

This project will be focusing on SMS fraud detection, and how we can implement AI & Machine Learning to further improve the detection methods and success rates. This project's aim is to further protect its users from this form of SMS phishing 'Smishing' and mitigate this threat as much as possible.

From my research and learning, I will be developing a prototype application. This application will display how much AI & ML could greatly improve SMS fraud detection.

The main purpose for this project's creation is due to the massive amount of fraudulent content that is spread online, specifically targeting users through SMS messages. To combat this threat, the process of providing Machine Learning to cross-check and scan SMS texts would greatly reduce the number of victims.

In conclusion, this application was able to receive, process, and provide predictions on the SMS messages. With a high success rate this application proved that it would be extremely useful if fully developed and available for users. Proving the ability for users to easily check their unknown messages would create a much safer environment when using their mobile devices.

Declaration

No part of this project has been submitted in support of an application for any other degree or qualification at this or any other institute of learning. Apart from those parts of the project containing citations to the work of others, this project is my own unaided work. This work has been carried out in accordance with the Manchester Metropolitan University research ethics procedures and has received ethical approval number Your EthOS Number.

EthOS Number: 2024-64066-48590
Project ID: 64066

Signed: Robert Hughes

Date: 17th May 2024

Acknowledgements

I would like to thank my supervisor Mohammed Al-Khalidi, who supported me through the whole project. Mohammed was always prepared to help and support me throughout this project.

Abbreviations

SMS	Short Message Service
AI	Artificial Intelligence
ML	Machine Learning
SMSC	Short Message Service Centre

Chapter 1: Introduction

SMS fraud has been consistently active around the world for decades. This form of attack is also known as ‘smishing’ which is the connection between SMS and phishing. The way this works is when an attacker sends a fake SMS message to misguide the receiver into giving up personal information or money. For example, the sender may pretend to work for the bank, requesting the receiver to share their passwords and or card details. These messages can also contain malicious links, that can spread malware onto the users’ device. These messages to many people will seem frighteningly real and leads to thousands of users falling victim.

Because of the relevance and constant use of this scam by attackers, this project aims to offer users a safe way to receive and scan SMS messages. To do so the application will classify and ‘judge’ these messages using Machine Learning, which will scan the received messages and warn the user of any suspicious activity or language it has detected. During this project I will be researching and testing different techniques to provide better security for users in the future. This report will also include details on how this prototype could be improved, with more resources and tools available, as well as further development notes and advice.

1.2 Background

This project’s research and development will focus on SMS fraud / phishing, how this works, and how we can detect and prevent this threat in the future. With instant messaging being so accessible everyone can message everyone, because of this access, attackers abuse this service to try and manipulate users. Throughout this report we will be looking at AI and machine learning techniques that could be implemented to detect, warn, and prevent users from falling victim to this attack.

SMS scammers often have very similar techniques and tactics to trick users into sharing information or clicking harmful links. By preparing a large dataset that defines Spam/Ham (Spam is malicious, ham is safe), the ML model will gain an understanding of what spam looks like, common language used, and how to identify it. From this training on the model, the application will then be able to judge user provided messages and determine whether it is spam or ham.

1.3 Aims & Objectives

The aim of my project is to create a functional alert notification system which can accurately classify fraudulent and safe SMS messages using machine learning. A secondary aim for this project is to create a user interface with good usability and accessibility features to ensure a smooth user experience.

The objectives of this project are:

- Create a functional prototype that will be used to test different machine learning algorithms and methodologies.
- Design, train, and implement a ML Model which produces a high classification accuracy score.
- Produce the final SMS Fraud Detector application with a high-quality user interface that produces accurate results to the users.

1.4 Learning Outcomes

This project perfectly aligns with my degree as I will be able to display the key principles of cyber security risk management, researching attacks/vulnerabilities, and finding and implementing mitigations. From this project I will be demonstrating my understanding of risks and controls, and essential analysis techniques during vulnerability assessment.

Another learning outcome from this project's completion is to understand methods to identify cyber threats, vulnerabilities and attacks and implement mitigation mechanisms.

1.5 Report Structure

2 literature Review

This chapter documents the research that has been done, including the background on this issue, the importance of this issue, as well as similar projects and solutions. This chapter's aim was to research what current practices are in place, and what could be adopted for this solution.

3 Product Requirements

This chapter contains the functional and base layer requirements that is expected from the SMS Fraud Detector. As well as the user requirements for this application.

4 Design & Development

Here the user interface designs are made, with key accessibility considerations. As well as the systems architectural designs, showcasing how the application will handle the user submissions and machine learning processes.

5 Implementation

Final decision for Machine Learning algorithms to be implemented, as well as software and development techniques to be used for this project.

6 Testing

How the application is tested to determine the success and accuracy of the application when meeting the aims and requirements for the project.

7 Evaluation

Looking back on the initial project aims and objectives to see how successful the project was, and what has been discovered through the testing stage.

8 Conclusion & Future work

Conclusion of the project overall, my personal reflection, and further project development suggestions that should be done in future work.

9 References

Chapter 2: Literature Review

2.1 Background

Short Message Service (SMS) is carried out using very few steps. Firstly, the user creates a text message on their devices, along with the recipient's number. This message is sent to the Short Message Service Centre (SMSC) where the server will handle and distribute the received SMS messages.

With modern-day technology, almost everyone has a mobile phone, which gives users a variety of essential communication methods to contact anyone anywhere. Due to this availability, there are many targets available to these scammers. These privacy threats can be especially harmful for vulnerable or older users who may not know how to spot suspicious activity like spam, fraud, or malicious links.

Malicious activity can be found in all online service areas, and this means that better, up-to-date software and tools need to be put in place. From prior research articles looking into global SMS traffic, it is found that in just 2016 alone, there was a total SMS traffic of over 1 billion messages (Portio Research Limited, 2016). This statistic is quite alarming as this traffic will have grown since 2016. Alongside this it is also worrying in terms of fraud, as if we imagine that just 0.1% of the 1 billion messages were spam, it will result in 1 million SMS fraudulent messages in circulation.

2.2 The importance of this issue

This project is very important because this type of attack is taking place constantly, all around the world due to its ease of access. Attackers can simply mass send these SMS messages using botnets and malicious applications to try and lure victims in with prizes, offers, or even threats.

Despite the ongoing research and actions taken against SMS fraud, the amount lost to these SMS scammers has reached over $330 million, as of 2022 in just the US alone (Salman, et al., 2024). This is a significant amount of money and clearly displays that this prevalent attack is in desperate need of more attention and counter measures.

2.3 Similar Research & Projects

- From one research project that presents an application for automatic smishing detection, they developed an app that would intercept incoming SMS, saved in JSON format, and forward this message to the rule-based detection model (Akande, et al., 2023). The rule-based detection and model development for training this application was very impressive and resulted with an accuracy of 98.42%. This automated SMS fraud detection could clearly help users identify fraud.

  While this research project was very successful, I believe that many users may not be comfortable and happy for their messages to be monitored and scanned automatically. Which is why for this project I am aiming to create a platform where users submit their SMS messages to be scanned as they wish. By offering a platform where users can simply input their messages that they are curious about would make them feel much more comfortable.

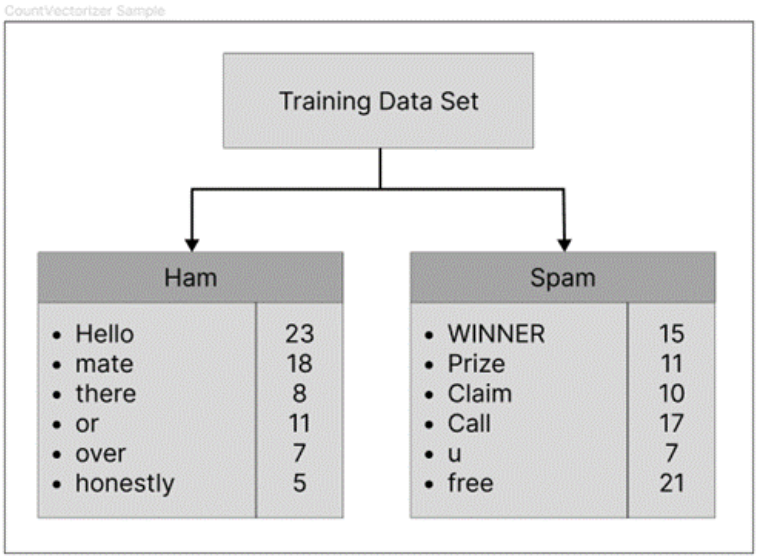
- Another similar research project I found was a different approach to classifying SMS fraud using Hybrid Machine Learning Models. This type of model was split into 2 stages, the first is the classification of the SMS message using a hybrid model, and other stage is the examination and security checks on its contained URLs. This hybrid model made use of 3 classifiers: Multinomial Naïve Bayes classifier, Extra Tree classifier and Random Forest. These are commonly used and high performing text classifiers and clearly worked well, with their recorded results of 96.86% accuracy.

  From this research project and its use of the Random Forest algorithm, it was clear that using this algorithm to train the model produced high classification accuracy. From this research article I have gained good understanding of the best common practices and algorithms to use in this area, and I will now be able to better plan what algorithms and technologies to introduce into this project.

Reading and looking into similar research articles has taught me about many different classifiers and techniques that I could incorporate into this project. This prior research shines a light on algorithms, techniques, and methods that other projects have incorporated, and what techniques could be used for this project.

2.3.1 Count Vectorizers

Count vectorizer is a command method used in natural language processing (NLP), this technique converts the learning text into vector-based data. This numerical data will state how many times each phrase / word has been used in the dataset (Singh, et al., 2022). This technique can help identify these SMS spam messages using this flagging system, to identify these keywords in potentially malicious messages.



(Figure 2.1, Count Vectorizer process.)

From **Figure 2.1** we can see that when performing this technique on just a small amount of data, it gives us a clear insight on what words are commonly used in fraudulent messages. With this we can then use the common spam words as 'trigger words' and use this as a method of detection when fed into the machine learning process. Using the count vectorizer technique would help identify commonly used fraudulent language when making predictions.

2.3.2 Naïve Bayes Classifier

Naïve bayes is a classifier that calculates its results based on probability, this is used for text classification and has been very useful in E-Mail / SMS spam detection in a variety of similar research projects. Naïve Bayes works on a 'bag of words' process, associating certain words with, in this case, spam / ham. Learning from large datasets, this classifier presents the user with predictions based on probabilistic results.

The Naïve Bayes classifier requires a set of records, which it will then use to create a frequency table. This table determines the number of times a word or attribute is used in the data set. The Naïve Bayes classifier will then create a likelihood table, which will contain words and phrases that are often used in fraudulent or malicious messages (Ghosh & Senthilrajan, 2022). This will then be used to make predictions on the SMS message that a user provides, to check for fraudulent activity.

In one related research study, they trained the Naïve bayes classifier using a detailed dataset of spam/ham messages. Once this model was generated after being trained, they were able to then test this model to see its success rate. The process was as follows: Receive Email, process Email, scan with Naïve Bayes Model, receive result, block / allow (Ma Ma, et al., 2020). The tests that followed resulted in very high precision results, scoring from 87-92% accuracy.

2.3.3 Fast Forest Ova (One-Vs-All) ML Model

Fast Forest Ova is a machine learning algorithm that is an iteration of the Random Forest algorithm. This algorithm is an optimized version of the Random Forest algorithm and provides results much faster when working with complex and noisy data.

As a One-vs-All model, it works through each class having their own binary classifier. Using this project as an example, the results be in binary form, Boolean 1/0. So here, spam would have a result of ‘spam’ or ‘not spam’, and Ham would have a result of ‘ham’ or ‘not ham’. These decision trees would then converge, and the result would come from the aggregation of the classifier outputs. While there are little research articles in Fast Forest, there are many tests done on the Random Forest algorithm. During a research paper, they compared popular ML algorithms like SVM, Naïve Bayes and Random Forest. This research resulted in Random Forest having the highest precision and accuracy (V P, et al., 2023).

2.4 Conclusion of Related Work

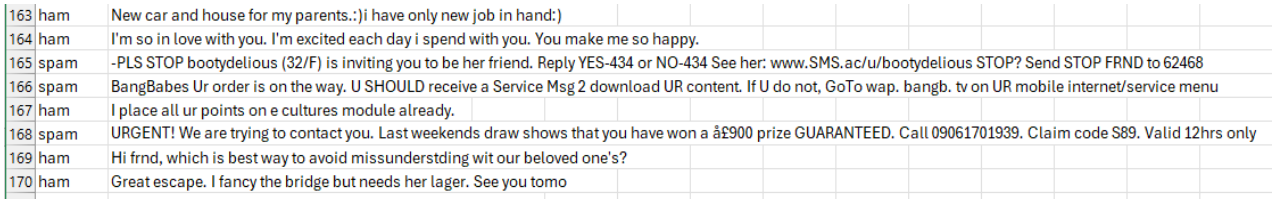
In conclusion, the extensive review of related research articles and projects has provided a good insight into the current methods, tools and techniques used in the current development of Machine Learning and SMS fraud classification.

This investigation of related work allowed me to learn about FastForest and ‘bag of words’, which will be key features in this project, and could improve the classification and identification of SMS fraud. The related work research also allowed me to find helpful data and statistics that support the use of the FastForest Ova showing its success rate, classification results and efficiency in related software projects.

2.5 Training Material

For the training material I will be outsourcing this content, using an online readily available SMS Spam dataset. Doing this will make it much easier to start the development process and train the model to understand, learn, and differentiate between spam / ham. For this I will be using a large dataset of over 5,500 labelled spam / ham SMS messages, which the Model will use to learn from. This dataset was provided by Kaggle, which is a reliable Machine Learning and Data Science community, which provides lots of tools and resources to do with ML.

Using an online public dataset may have issues such as low-quality data, or inaccurate samples for the model to learn from. However, with this project being a prototype, it will save a lot of time by using a readily available data set to get started. If this project was to be taken into further development, a larger, and higher quality dataset would need to be made / used.



163	ham	New car and house for my parents.:)i have only new job in hand:)
164	ham	I'm so in love with you. I'm excited each day i spend with you. You make me so happy.
165	spam	-PLS STOP bootydelious (32/F) is inviting you to be her friend. Reply YES-434 or NO-434 See her: www.SMS.ac/u/bootydelious STOP? Send STOP FRND to 62468
166	spam	BangBabes Ur order is on the way. U SHOULD receive a Service Msg 2 download UR content. If U do not, GoTo wap. bangb. tv on UR mobile internet/service menu
167	ham	I place all ur points on e cultures module already.
168	spam	URGENT! We are trying to contact you. Last weekends draw shows that you have won a â£900 prize GUARANTEED. Call 09061701939. Claim code S89. Valid 12hrs only
169	ham	Hi frnd, which is best way to avoid missunderstding wit our beloved one's?
170	ham	Great escape. I fancy the bridge but needs her lager. See you tomo

(Figure 2.2, Sample of the training dataset used.)

The image above displays just a small snippet from the total dataset of over 5,500 spam / ham messages that will be used to train the model. This will train the ML model on what spam is, common words that are used, language, idioms, etc.

2.6 Evaluation Techniques

Throughout the whole development process, I will be regularly testing the software's functionality and accuracy, as well as the usability and accessibility to ensure its easy to understand for all users. These tests will allow me to find errors and bugs early on and fix them quickly before the development phase has progressed. Continuous checks and extensive testing ensure that the application is high quality and fully functional. This also helps me plan further ahead and work towards the deadline for this project.

2.7 Ethical issues & concerns

For the development of this project the only ethical issues that could occur would be within the dataset that we are to use when training the ML model. Because of data protection regulations we must ensure that the training data doesn't have any personal of private data and information.

Despite this, there are no other ethical issues or concerns that need to be addressed. This is because for this project's development and research, we won't be involving any participants to test the app or provide feedback. For this, all the testing and final checks will be done by me throughout the applications development, and towards the end of the project's deadline.

Chapter 3: Product Requirements

To create a successful application, meeting the requirements of the application is vital. The requirements should state what this app will provide, what services, libraries and tools will be featured. To make these choices, considerations of user requirements and research statistics / results is necessary to produce an app that solves this ongoing issue.

3.1 Functionality Requirements

To understand the functionality requirements and features that this app will contain, we need to revise the problem statement that I wish to address and minimize. The problem that mobile phone device users face is SMS fraud, also known as ‘smishing’. This is a problem that is commonly known among all types of telecommunication devices. Viewing this problem as a user or potential victim allows us to understand what type of tools they need to stay safe from this fraudulent activity.

3.1.1 Requirements Discovered

1. **User entry** – User submission for potential SMS fraud where they can submit received messages that they are suspicious or wary of.

2. **SMS Scanning** – This feature is the most important part of this application, providing a good machine learning model to scan these messages for fraudulent or suspicious activity.

3. **Feedback & Advice –** Provide the results from the scan, alongside a short explanation of the results as well as advice on how to handle the situation.

These requirements are crucial towards the success and functionality of the application.

3.2 Base Layer Requirements

Requirements Discovered:

1. **Accurate Classification** – Precise judgment is essential for this application to succeed and keep users safe. Rigorous testing of the application would result in valuable insights into the quality of the machine learning model and its decision-making. As if this application cannot produce accurate results, then it is useless.

2. **Quality UI/UX Design** – Ensuring the application is aesthetically pleasing, easy to understand, and provides a good user experience. This is important as without considering this, the app may be inoperable or hard to navigate for users.

3. **Performance** – To ensure a good user experience and keeping users satisfied, fast response times are very important. As without these considerations, users may become agitated and resort to other more optimized applications.

These background requirements also play a large part in the development process and need to be considered when designing and coding the application.

Chapter 4: Design

4.1 Introduction

Throughout this document, research, planning, and investigations have been carried out to understand this sector further, and learning about the current tools and methodologies used to produce such an application.

The design phase of the application is a very important stage in a project's lifecycle. "Design is the only way to accurately translate a customer's requirements into a finished software product." (Singh & Gautam, 2016). Designing the system structure and architecture is important to plan, understand and showcase how this system will work overall, with all features, elements, and user requirements considered.

4.2 Approach

The priority of the design approach is to ensure that this application is user friendly and accessible for all users. Following common design guidelines from reputable sources such as Apple and Microsoft provide key research evidence on how to build and design simple, easy to use and understand applications.

To do so, looking into similar available applications and their designs can help visualize good potential design layouts, as well as improvements that could be made. To further ensure that this applications design is accessible, choosing large, easy to read fonts is important to ensure that this application meets the user's requirements. To begin the design stage, simple wireframe designs, system architectural designs, and process diagrams need to be created to understand and plan how this app will look, operate, and process information. From this practice, errors and oversights can be resolved before beginning the implementation.

The application's UI will be created using visual studio, connected with the Machine Learning Model Builder to train and build the machine learning model.

4.3 User Interface Design

Previously mentioned in the introduction and approach, it is very important to plan this UI design, with a range of considerations like usability and accessibility. To do so, following a simple and clean UI design will be done throughout this process. Providing a simple user interface layout will improve user experience by making it more accessible for all users.

The following figure displays the mock-up designs that have been created for this application. With considerations of usability and readability.



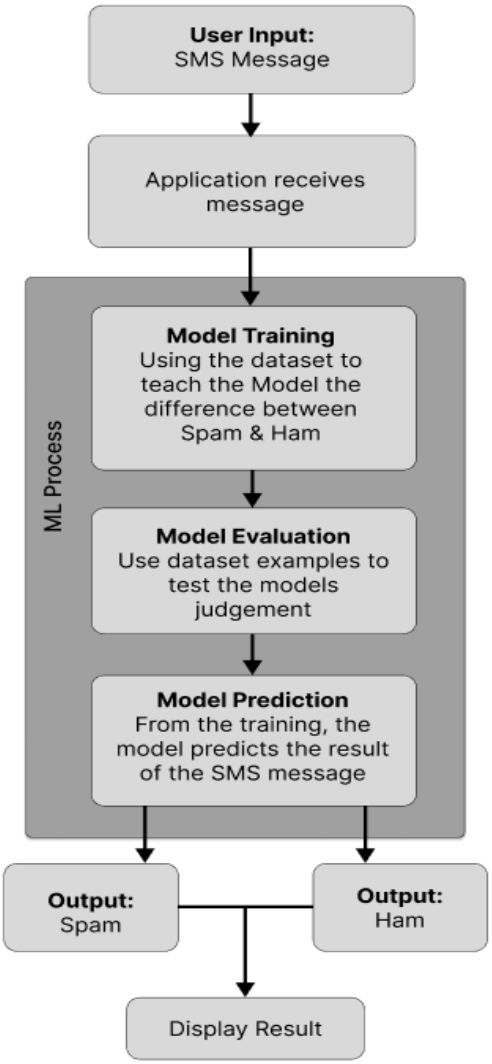
(Figure 4.1, Wireframe designs for the Application)

This wireframe in **Figure 4.1** provides a good base design to begin the initial development of the application. This also allows me to evaluate the overall layout of the application and make any changes or improvements before beginning the development process.

From this main page of the application, users are provided with a simple, clear layout,

alongside instructional information to get started. The user will be able to enter their SMS message in the entry and select the ‘Scan Message’ button. From here the ML Model will process the input and display its result and explanation.

4.4 System Architectural Design



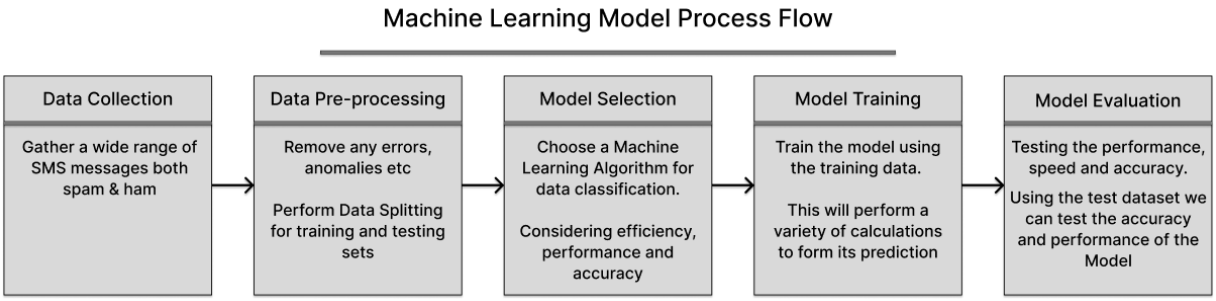
(Figure 4.2, Application Flowchart Diagram)

Figure 4.2 displays the design and structure of how the application aims to operate, and process information. Due to the main function of this app being a successful prediction algorithm, so planning and problem solving for this stage is very important. Doing this will also allow issues or potential dependencies to be resolved and figured out before the coding and deployment begins.

Creating an appropriate flowchart diagram for the application's operation will involve all processes from the input to the displayed result. During this design stage, many different versions were updated and improved to ensure the process design will be efficient and high performance.

As seen in the diagram above, the application will receive the user input, and then refer to the machine learning model. The ML model will be trained using a large dataset of fraudulent SMS messages, as well as safe messages. Using the Visual Studio Machine Learning model builder, we will be able to choose the highest performing algorithm, train the model, and then implement it into the application. From this training, the model will be able to understand what fraudulent and suspicious language looks like. Based on this training, the model will make accurate predictions on the SMS inputs and display the results to the user, alongside advice and its result explanation.

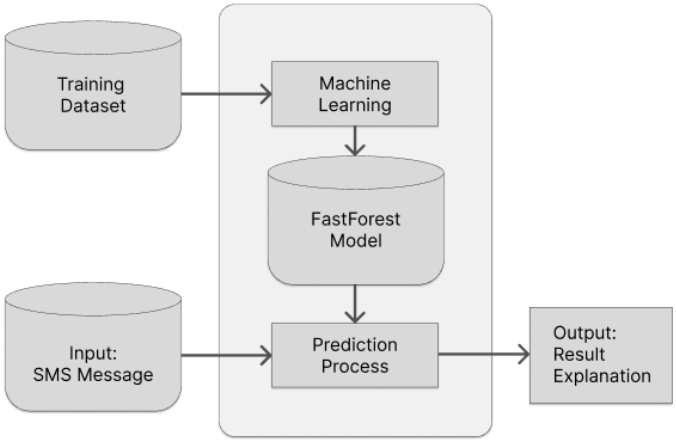
4.4.1 Machine Learning Model Design



(Figure 4.3, ML Model Selection, Training, and Evaluation)

To create a high performing machine learning model, many factors and stages are involved before this can be implemented. Firstly, I gathered a large dataset that includes both spam & ham messages, and then processed the dataset to ensure its good quality and does not contain errors or inconsistencies. The software will then train the model, using half the dataset as a training set, and the other half as a testing set.

4.4.2 System Component Design



(Figure 4.4, System Component Design)

From looking into the classifier algorithms visual studio offers, Fast Forest Ova stood out to me due to its speed and performance. As an improved version of the Random Forest Algorithm, this model performs highly when working with Boolean classification, producing quick and accurate results.

When evaluating the model with the test set, it resulted in an accuracy score of 89%. This impressive accuracy of the model means that it will perform highly for the users, and most importantly, provide accurate feedback.

4.5 Project Timeline Plan

To ensure the project's success, implementing a good timeline plan is important in the design phase of the project.

- **Stage 1: Design & Planning**

The first stage, as seen above, includes the creation of the wireframes, machine learning diagrams and the system component designs. This helps me better plan the application and future work that is to be done. Understanding how to start the implementation stage is crucial towards the success of the project.

- **Stage 2: Coding & Implementation**

  From the prior research and planning, the implementation stage will begin. Developing a high-quality user interface, following the wireframe design plans with ease of use and accessibility considerations in mind. When coding the application, a lot of trial & error, as well as research will be done to be able to create a successful SMS fraud detection software. The main element I wanted to focus on for this application was the machine learning model accuracy, how good it was at classifying SMS messages and evaluating the language used.

  The other key element of this application is how it will operate, by taking input from the user, they get to choose what messages are scanned and doesn't mean that all the users' messages are automatically read by the app. I believe this was the best direction for the application as it meant the users felt more in control with this choice.

  The user interface, whilst still important for a successful app, was not the main focus during this stage to ensure that if there were any time constraints deadlines would still be met.

- **Stage 3: Testing**

  Code Evaluation – This method involves the process of going through the program's code, checking for errors, defects, or unused lines of code. This process helps refine the code and correct any errors. This process also included unit testing, testing all the low-level items like buttons or input fields. This is beneficial as it means the code is concise and easily understandable, as well and ensuring that all elements work.

  Performance Testing – Testing the applications performance is also very important for this project, as the results produced must be accurate if this app is to be deployed. This is because users would trust this application to check their SMS messages for fraud or spam, so it is vital that it displays accurate results.

  The main method employed here is that when the dataset is split into training and testing sets, we can get an accuracy score instantly based on the results from the testing data. We can then record the success rate from this performance test and see how effective this ML Model is at identifying SMS fraud/spam messages.

Accessibility Testing – Testing the software's accessibility is also a very important testing process when evaluating the app. This process consists of checking that all the text, buttons, and any other content is easily readable and understandable for all users, considering how users may suffer from impaired vision of dyslexia. To ensure this, large font sizes will be used across the tool, making it easier for all users to read and understand. With this, clear fonts will also be used across the app to establish good readability.

"Delivering any software product with best quality is the main goal of any project. Which means testing plays a very important role in development of any software product." (Sneha & Malle, 2018). Overall, testing is one of the most important stages to ensure the project's success and functionality.

- **Stage 4: Evaluation**

At the final stage of this project's development will be the evaluation, which will be done on the finished product. This evaluation will help check for any things that could be improved, or if there is anything missing from the project. This evaluation will cover all aspects of the application, including performance, efficiency, functionality, and overall software quality. This is important to judge my application, what was done well, and how this project could be improved and done better in the future.

4.6 Equality, Diversity & Inclusion

Design Inclusivity Considerations

When designing and creating the user interface, it was important to ensure that all elements of the UI were accessible to all types of users, with different disabilities and impairments. Apple's mobile device guidelines states that using large font sizes like 17pt+ is key when aiming to provide good usability and readability for all ages in a mobile application (Apple Inc, 2023). From these guidelines, font sizes of 17-18 were used for all important elements such as entry fields, instructional information, button controls.

Employing large easily readable fonts made this application much more inclusive and user friendly. By doing so, the application ensures that no users are left out or struggle operating and understanding the app.

Diversity & Equality Considerations

When designing and coding any application it was crucial to consider that many users can suffer from a variety of different impairments and difficulties such as vision impairments, dyslexia, or learning difficulties. By providing large fonts and a simple, easily operable user interface, it increases the usability and overall user experience.

Chapter 5: Implementation

5.1 Introduction

This Chapter will cover the final designs, features and software choices made for the application. In this chapter, the development of the code will be documented, along with the creation of the UI, and any challenges that came to light during this process.

The development of the project will begin following the agile methodology principles, which is proven to help produce high quality results. “Adopting agile development methodologies has a positive impact on both the productivity and the quality.” (Kumar & Bhatia, 2012). The Agile methodology promotes good testing and evaluation checks through iterative processes, and therefore adds good error/fault detection to the project.

5.2 Machine Learning Implementation

The Implementation of the ML is very complex but also one of the most important components towards the success of this project. To do this I will be using the Visual Studio Machine Learning Model Builder.

5.2.1 Dataset Implementation

0	In sch but neva mind u eat 1st lor..
0	Hey whats up? U sleeping all morning?
0	Erm. I thought the contract ran out the4th of october.
0	I dunno until when... Lets go learn pilates...
1	U are subscribed to the best Mobile Content Service in the UK for å£3 per ten days until you send STOP to 83435. Helpline 08706091795.
0	Yup i'm elaborating on the safety aspects and some other issues..
1	3 FREE TAROT TEXTS! Find out about your love life now! TRY 3 FOR FREE! Text CHANCE to 85555 16 only! After 3 Free, Msgs å£1.50 each

(Figure 5.1 Spam.csv dataset)

The source of this dataset was Kaggle, which is a data science community that offers a range of tools and resources. This dataset contained over 5,500 labelled SMS spam / safe

messages, which will be used to train and test the machine learning algorithm. Using a large dataset allows for the model to understand and classify SMS messages better and improve its classification accuracy.

5.2.2 Data Preprocessing

To ensure that the data is clean, accurate, and ready to use and train the model, preprocessing is required.

1. **Data Cleaning** – Removing any empty or null values from the dataset, this is important so that there are no null labels or message inputs fed to the ML model.

2. **Text Processing** – This is done on the message data. Here, all the text is converted into lowercase characters, and tokenization is automatically done where spam words and keywords are tokenized, to be used to identify fraudulent SMS.

3. **Feature Extraction** – Extracting features from the raw data such as: Bag of Words, keyword lists for keyword filtering, and common features found in the fraudulent messages.

5.2.3 ML Model Builder

The data preprocessing is done to ensure that this data is error free and ready to be used to train and test the ML model. This preprocessing of the dataset is done by the Visual Studio ML Model Builder automatically. This tool was one of the reasons that I chose VS to create this application.

The preprocessing stage comes with many benefits towards the final model. This includes improved model accuracy and performance. By selecting and identifying relevant features the model can identify spam from ham using common patterns or keywords found in the messages.

“The choice of preprocessing techniques can greatly affect the model performance and preprocessing selection by the user can very often be sub-optimal as the user cannot directly explore all preprocessing techniques and their combinations.” (Mishra, et al., 2020)

This research into preprocessing data and its advantages shine a light on the benefits and best practices when preprocessing the data for the machine learning algorithm.

FastForest Ova Algorithm

Using the ML Model Builder, it offers many different machine learning algorithms to choose from. When testing different algorithms, it resulted in the FastForest Ova performing best with the highest accuracy.

After the model builder training and evaluation, the implementation of the ML was started. See Appendix C (Prediction Code) to view the code behind the prediction process, where it displays how the user input is fed to the model.

```
1 reference
public async Task Scan()
{
    // Setting up relevant variables and setting the busy state to true.
    string ResultLabel;
    string ResultExplanation;
    IsBusy = true;

    PredictionResult = await Task.Run(string () =>
    {
        // Here the user ML 'message' is set to the user entered message,
        // and given to the SpamDetector Model as a new input for it to predict.
        var sampleData = new SpamDetector.ModelInput()
        {
            Message = ResultText
        };

        // This if statement then determines the user output from the boolean results.
        // If the predicted label is 1, set the result label to 'Fraud' including its explanation
        if (SpamDetector.Predict(sampleData).PredictedLabel == 1)
        {
            ResultLabel = "Fraud\n\n";

            // Defined in Synoptic/ML/Explanation.cs
            ResultExplanation = Explanation.GenerateExplanation(ResultText);

            // simply connecting them to one output to return
            return ResultLabel + ResultExplanation;
        }
        // if result is not 1, then it is safe.
        else
        {
            return "Safe \n \n While this messages seems safe, never share personal information," +
            " or transfer money to unknown users.";
        }


    });
    // Process finished, busy = false and the user can now scan another message.
    IsBusy = false;
```

(Figure 5.2, ML Model Input)

When coding the Scan task, it had to communicate with the model by sending the user input as a new prediction, which would then return the result. As seen in the **Figure 5.2** above, the input is given to the model, and based on the returned Boolean results, the appropriate output is displayed to the user.

5.3 Software, Algorithms, and Techniques

Visual Studio 2022

After considering many applications and software tools, Visual Studio 2022 appeared to be the best choice for making applications and integrating machine learning. This software also stood out as it offered many reliable tools like the ML Model Builder, and a range of community plugins.

Visual studio was used to create the .NET MAUI (Multi-Platform App UI) application. This tool also provided a Machine Learning Model Builder, which was used to easily create, train, and implement machine learning into application.

Fast Forest Ova

For this project, the FastForest ML algorithm was selected due to key research proving its success and accuracy. FastForest is an optimised, faster, and more accurate algorithm that has stemmed from the Random Forest algorithm. In a research study into FastForest, it found that it was 24% faster than Random Forest and proved that in these experiments FastForest beat Random Forest in most classification accuracy experiments (Yates & Islam, 2021).

This model utilises ensemble learning, where there are multiple trained models under one primary model. These decision trees will all predict and classify the message individually. The aggregation process is then performed to produce the final prediction results, which will be displayed to the user. This approach using ensemble learning is proven to be much more accurate over single decision trees (Microsoft, n.d.).

Keyword Based-Filtering.

Using a large set of keywords commonly used in SMS fraud can be helpful at identifying fraud, and creating an output that provides the user with an explanation of the result. This will be used in the application to explain the result, for example: “This message has been flagged as fraud due to the suspicious language used: Free, Win, Money, Claim now, Urgent.”.

5.4 Code & Challenges

See Appendix B: User Interface Code (Main Page), which shows the code behind the user interface, alongside the layout and binding elements and variables.

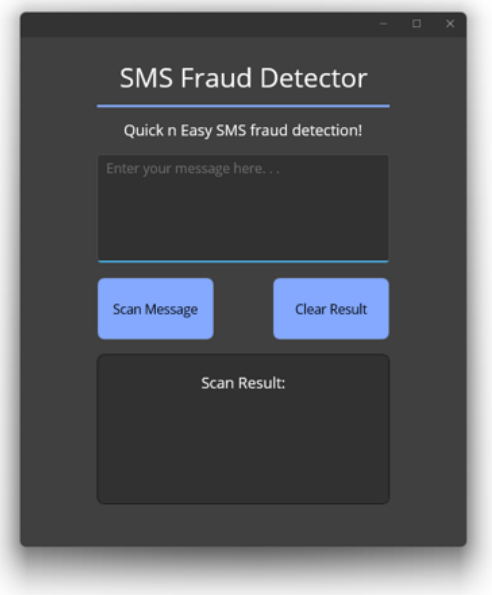
5.4.1 User Interface

Creating a clear and easy to understand user interface was important to ensure that all users could successfully operate this application easily. Appendix B specifically displays code behind the main page UI.

This part of the development process was simple, however still important. Creating a simple user interface allows for much greater accessibility for users. With this the UI has been designed to ensure that all text content makes use of contrasting colours (white on black / grey) to ensure better readability for any users with limited vision.

When working on the UI with disabilities in mind means that any users who suffer from these difficulties will have a better experience and understanding of the application. However, this will also benefit all users. “Although accessibility features are designed mainly to aid people with disabilities, they can provide a competitive advantage and potential benefits to all users, as they can make everyone more productive.” (Kavcic, 2006).

Final User Interface



(Figure 5.3, Final UI Results)

This final version of the applications UI adds great value to the app as it provides users a simple, clear, high-quality interface with comprehensive instructions and controls.

Interface Functions & Features:

The front-end main page for this application is very simple, however, the key elements and functions on this user interface allow for good control and navigation. These elements are listed below:

1. **Instructional information** – Providing the users with instructions and helpful text makes it easy for users to understand how to use the application.

2. **User Entry** - To allow the user to submit their SMS messages for processing, it is important to provide a user entry field. In this field the user can enter the message that they are unsure of and want to get feedback from.

3. **Scan Button** – When clicked, this will process the user input, send it to the ML process and the explanation generator. Once the result has been finalized, the application will display the result and explanation to the user.

4. **Clear Button** – To help make the application easy to understand when scanning new messages, this button, when clicked, will clear the output text area where the results are displayed.

```
1 reference
public async Task Scan()
{
    string ResultLabel;
    string ResultExplanation;
    IsBusy = true;

    PredictionResult = await Task.Run(string () =>
    {
        var sampleData = new SpamDetector.ModelInput()
        {
            Message = ResultText
        };

        if (SpamDetector.Predict(sampleData).PredictedLabel == 1)
        {
            ResultLabel = "Fraud\n\n";
            ResultExplanation = Explanation.GenerateExplanation(ResultText);
            return ResultLabel + ResultExplanation;
        }
        else
        {
            return "Safe \n \n While we have no noteable discoveries, be wary about " +
            "certain activity and always contact support lines before taking action.";
        }


    });

    IsBusy = false;
}
```

(Figure 5.4, Communication with the model)

The coding process posed challenges when working with the ML model when trying to run the SpamDetector process and receiving accurate results. To overcome these struggles, a lot of time was invested into learning and practicing how to use Machine Learning, especially within Visual Studio .NET MAUI.

From overcoming these technical challenges with a fully operation ML process, the application was able to receive the user input and swiftly display results to the user. The explanation factor of the result was also provided to give the user better insights into how the result was produced.

5.4.2 Explanation Generator using Keyword Filtering

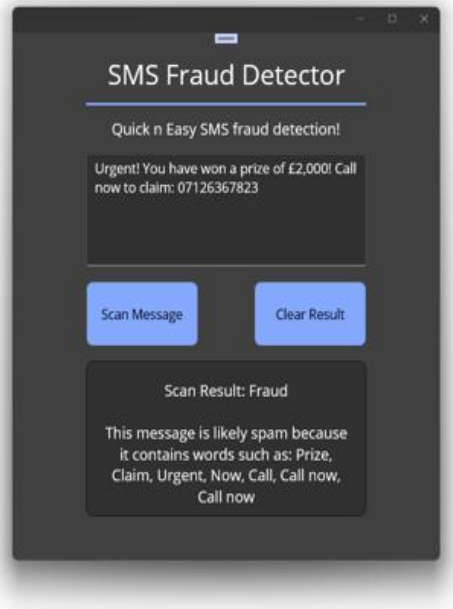
```
public class Explanation
{
private static readonly List<string> SpamKeywords = LoadKeywordsFromCsv();

1 reference
public static string GenerateExplanation(string message)
{
    var containsSpamWords = SpamKeywords.Where(keyword => message.Contains(keyword,
        StringComparison.OrdinalIgnoreCase));

    string explanation = "This message is likely spam because it contains words such a

    if (containsSpamWords.Any())
    {
        explanation += string.Join(", ", containsSpamWords);
    }
    else
    {
        explanation = "No specific reasons found. This message may not be spam.";
    }

    return explanation;
}
```



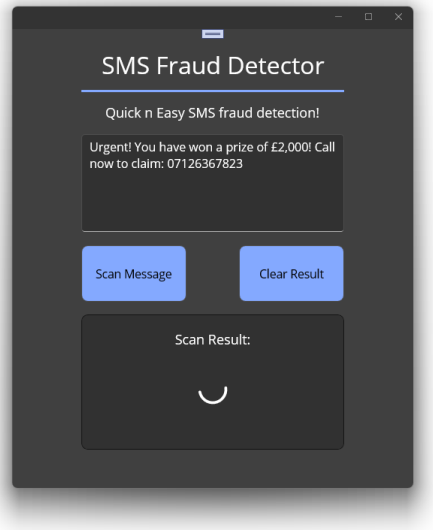
(Figure 5.5, Explanation Generation Process)

This explanation class uses a large dataset of “BadWords” or fraudulent words, to scan the message for. This will be connected to the result displayed if the prediction outcome is fraud and will give the user a better understanding of how the language used in the message raises suspicion.

This Features was created using keyword filtering. To do this I used a dataset that contained suspicious spam words that are commonly used in fraudulent SMS messages. This would then scan the received messages and save the found keywords to a list, which would then be displayed to the user to explain how the language used has produced this result. This process can be seen in **Figure 5.4** above.

5.4.3 Error Handling

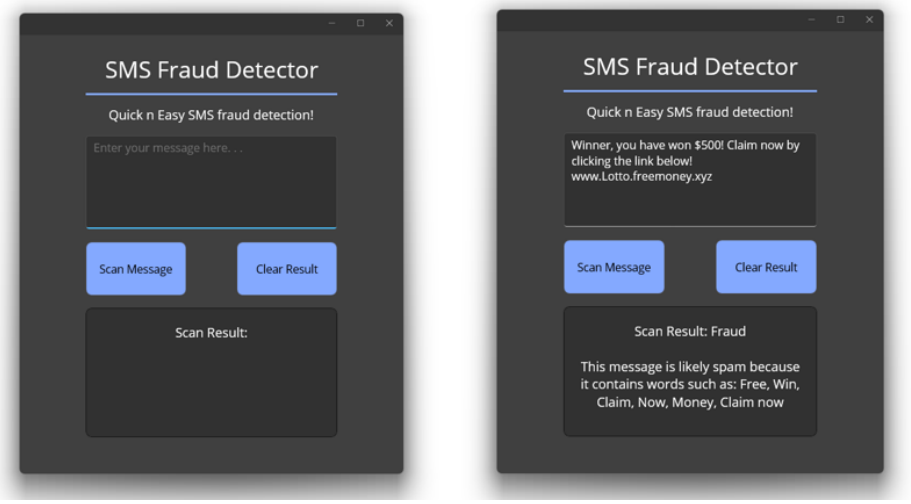
To overcome basic input errors with the user running the model whilst it is busy, a “IsBusy” function was added to ensure that it could not be processing more than one message at a time. This simple error handling function proved itself as a good mitigation to prevent the application being overloaded or overworked.



(Figure 5.6, Error Handling, Loading Indicator)

The figure above shows the loading effect that indicates the ML Model is busy processing the current request. This helps provide the user with visual feedback, notifying them that the application is still processing their request. This process also eliminates any issues with users submitting multiple requests at once and overloading the application.

5.4.4 Final Application



(Figure 5.6, Final app version)

The figure above displays the final version of the application. When the user submits their message, the ML Model will receive the input, and within seconds, create and display the

prediction and explanatory result, which can be seen in the example above.

Final Version Key Features

Whilst the UI and front end of the application is very simple, there were many complex background features and elements that were implemented during this stage of development. These are:

1. **Machine Learning Model -** Using the ML Model Builder to create, train and implement a robust and efficient ML Model to classify SMS messages.

2. **FastForest Algorithm** – Iteration of the Random Forest algorithm, offering high classification accuracy, with efficient and quick results.

    a. **Ensemble Learning** – Multiple models to improve the accuracy of its results.

    b. **One vs All** – Each label, spam or ham have their own binary classifier. Helps in ensemble learning to improve accuracy when aggregating predictions of multiple classifiers and outputs.

    c. **Probabilistic Results** – Probability scores over Boolean results of just true/false ensures better result accuracy, especially when voting.

3. **Keyword Filtering** – To feature an explanatory result to the user, scanning the message using a keyword filter to pick up fraudulent language can help display and explain why the message has been flagged as fraudulent.

Chapter 6: Testing

6.1 Introduction

When the implementation and development of the application is complete, testing the functionality and accuracy of the app is very important. Performing a wide range of tests allows me to discover any flaws or errors in the application, whilst also getting a good understanding of how accurate the app is.

The projects development has followed an agile methodology, which encourages a cyclical development process with constant testing of the application throughout. To follow this, testing was done after all feature implementations and updates.

When performing these tests, it is important to test all possibilities and types of input that the app may receive. To do so, planning and following a functionality table to test and document the inputs and outputs is important to evaluate the applications functionality.

6.2 Testing Methods

When testing the application, it is important to plan what areas and features of the application need to be assessed. To ensure that this application meets its aims, it is key that it provides a good user interface where accessibility needs have been considered. Testing for good UI/UX design will ensure that this application is easy to use and understand for all users.

Another testing area is the performance testing. This mainly involved testing the ML Model to assess its classification accuracy to see how well it can identify fraudulent messages. To do this, when training the model half of the SMS Spam / Ham dataset is used to test the performance. Here the ML model will predict each message from the dataset and then compare these results to the actual label.

The final testing type that will be performed is the functionality testing of the application. The table below displays and documents the tests that were done on the application, alongside the expected and actual outcomes. This is important to test on all user entries and controls to ensure that all elements and outcomes of the application function properly.

To test the application, I will be submitting a range of commonly used fraudulent messages, as well as normal safe messages. This will help me to test and determine how well this application operates, and how accurate the predictions are.

6.3 Testing Table

Area	Input	Expect outcome	Actual outcome
User entry	Empty / null	Error message of empty input	Error message of empty input
User entry	Suspicious Message	Notify user of fraudulent message, with explanatory message.	Notify user of fraudulent message, with explanatory message.
User entry	Safe Message	Notify user of safe message, alongside further advice about sharing data or sensitive information.	Notify user of safe message, alongside further advice about sharing data or sensitive information.
'Scan' Button	User clicked	Sends message to ML model & receive prediction result.	Sends message to ML model & receive prediction result.
'Clear' Button	User clicked	Clears the text editor box.	Clears the text editor box.

(Table 6.1, Testing Table)

From **Table 1**, displaying the functionality tests, we can see the success of aims met when comparing the expected outcomes with the actual outcomes. Meeting these targets ensures that this application is functional and operating appropriately.

In conclusion, the testing stage of this application was very successful, resulting in a fully functional application and meeting all the goals and objectives previously set out.

Chapter 7: Evaluation

When I first started this project, the aim was to create a fully functional software which could identify and classify spam / ham SMS messages using machine learning. This application aimed to offer users a platform where they could scan their messages and receive instant results, as well as a short explanation. This project also aimed to provide a high-quality user interface, accompanied with good usability and accessibility. The SMS Fraud Detector app has completed and met all aims.

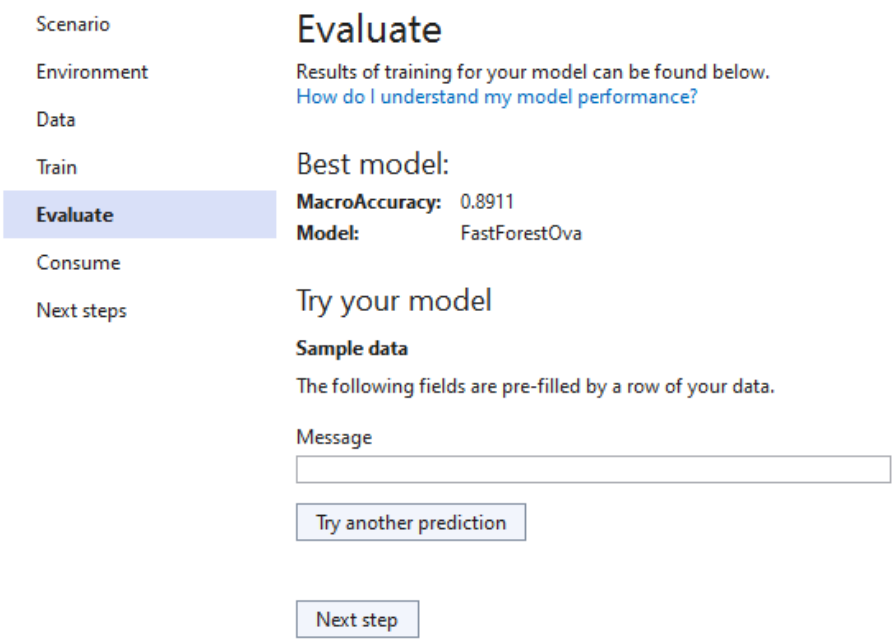
This ongoing issue that causes incredible amounts of financial loss everyday has been further mitigated with the development of this solution. This was resolved by providing a platform where anyone can submit their unknown messages and get accurate feedback results and explanations. Providing an application with a classification accuracy of over 89% meant that users could get instant, accurate feedback and help when dealing with potential SMS fraud. Producing a high classification accuracy in a project like this is incredibly important, as without good accuracy, many users could be lured in by potential fraudulent and malicious SMS messages. By achieving a high classification accuracy ensures that users will be better protected from future SMS fraud and therefore leads to a successful project outcome.

The usability and accessibility side of this product was also essential when working on this project. This is crucial as providing a highly functional and accurate application is good, but if many users find it confusing, hard to understand or difficult to operate, then as a software tool it has failed. The usability and accessibility design considerations can clearly be seen in the final product. By offering a clear, simple design makes it easy for all types of users to easily understand and use the application. The limited amount of information and controls on-screen create a sense of understanding and comfort for the users, making it much easier for them to use the app and not be overwhelmed.

7.1 Testing Evaluation

Because machine learning systems must make complex predictions and calculations, it is very hard to ensure that this software provides the correct answer every time. “Machine learning systems are difficult to test because they are designed to provide an answer to a question for which no previous answer exists” (Zhang, et al., 2020). This research into quality assurance with ML models provides a clear statement on the difficulty of training these systems.

To ensure that the software is highly accurate, half of the SMS dataset was used to train the model, while the other half was used to test the model’s performance. Because it has all the messages labelled as spam or ham, this allows us to easily test the accuracy and performance of the model. This testing and model evaluation is most common in machine learning and provides good insights into the accuracy of the algorithm.



(Figure 7.1, ML Model Builder Evaluation)

In the figure above it displays the evaluation page for the ML Model when using the Visual

Studio ML Model Builder. In this model builder it offers an evaluation page, where it will automatically use half of the dataset to test the accuracy of the model. For the final application, the SMS Fraud Detector was performing with an 89% classification accuracy.

Further evaluation and testing were conducted by simply testing all features of the application. Providing seemingly safe and suspicious messages to see how the application would respond, and the results it would provide.

7.2 Comparing to available solutions

Current Available Solutions

With the current available applications, there are 3 consistently found flaws. These are: Privacy concerns, poor user experience, and false negatives.

1. **Privacy Concerns** – A common issue I found with similar projects available was the disregard of privacy. Many methods, while productive and effective, would simply scan all incoming SMS messages, and scan them in the background.

   My approach was slightly different, allowing users to input their messages and get results in real time using the application. This strategy was taken to ensure that users continue to keep their messages private, whilst also being able to get advice and feedback on the messages they are unsure of.

2. **Poor User Experience** – Many applications found in the research period offered good performance and accuracy, however lacked in UI/UX design. Offering high quality UI & UX greatly improve user satisfaction and means that users will be more inclined to continue using the application. To ensure a good user experience, planning and mock-ups were produced, with extensive research into software design & development to solve this common mistake.

3. **False Negatives** – Another key issue with many solutions was the alarming number of false negatives. False negatives in a SMS fraud detection application could lead to users falling for these scams, due to the result or advice from the application. From trying out

web and mobile solutions for SMS fraud detection, false negatives were often produced, with some offering further advice, whilst others didn't.

To prevent the reoccurrence of this mistake, false negatives have been considered. In this project, further advisory steps are provided even when the results determine the message safe. This advice states that even though the application has deemed the message safe, it is always best practice to contact the organisation's help line, or the sender's original number, especially before ever sharing information or transferring money.

7.3 Final Evaluation

This application has been successful fulfilled the aims and objectives set out in the beginning of the project, and I believe that the final product is a great success.

Providing a platform that can automatically scan messages for fraudulent or suspicious activity successfully, could massively help potential users against fraud in the future. Which is why I believe that this app could be very successful.

Chapter 8: Conclusions & Future Work

8.1 Conclusion

The problem with the online world is that there will always be malicious actors out there, and the freedom and accessibility to communicate with anyone anywhere in the world can be abused by these individuals. Considering this fraudulent activity has been around for decades, its prolific operation will continue forever.

The SMS Fraud Detector has been designed to offer a great solution to this ongoing issue. Providing a highly accurate ML application where users can scan their SMS messages and get good feedback and results. This Fraud Detector has been carefully designed and tested to provide good usability through simplistic designs and easy to understand controls to ensure that all sorts of users can operate this application and remain safe online.

Reflecting on the coding and testing stages throughout the entirety of the project's lifecycle, I am confident that the aims and objectives set out were met. The final version of the Fraud Detector was able to successfully receive SMS messages, scan them for potential fraud or suspicious activity, and deliver accurate results adjoined with an advisory explanation. In addition, the user interface created for this application was user-friendly and easy to understand.

8.2 My Reflection

When looking back on the project, I think that it has very been successful. All the project's aims and objectives were met and the final product, whilst simplistic, was professional and performed exceptionally well. With limited understand and experience working with and creating machine learning models, the beginning of this project was daunting. However, with good research and online learning sources I was able to quickly learn how to develop, train, and implement ML within my application.

Whilst I believe that the result of this application was very successful, looking back I do wish I had more time to put into this application. With extra time, adding more ML processes and detection features would have been very beneficial to future users if deployed. Along with this, better training data and feature improvements towards the Model would be a priority.

8.3 Further Project Development Ideas & Suggestions

Real-Time Monitoring

Whilst this was not an objective of this application due to potential privacy concerns, allowing users the option to enable real-time monitoring would improve the applications productivity. Offering real-time monitoring would also improve the users experience, by automatically scanning incoming SMS messages, alerting the user when suspicious or fraudulent activity is detected.

By instantly and automatically scanning the incoming SMS messages that a user receives also means that all the user's messages are scanned. This would massively help mitigate the possibility of successful fraudulent SMS messages.

Multi-Platform Functionality

Whilst this product has been tested on both Windows OS and Android devices, providing this application to all device types would mean that anyone can access this tool and scan their messages for potential fraud. This would maximise accessibility and usability by offering the application to all devices. This would be very beneficial to the success of the application, allowing for better user accessibility, when providing this tool for all devices.

Improved Classification Accuracy

While 89% is a good classification accuracy score, this could be improved to ensure that the application is making as accurate predictions as possible. This further development would help boost the success of this project, by having far better fraud detection in place to protect the users.

This could be done by training the ML model with a larger dataset of SMS fraud messages, so that it has more examples of spam and ham messages, and therefore will be able to make more accurate predictions.

Despite these future improvements, this project has resulted in a very successful anti-fraud tool which could help and save many people from great financial loss.

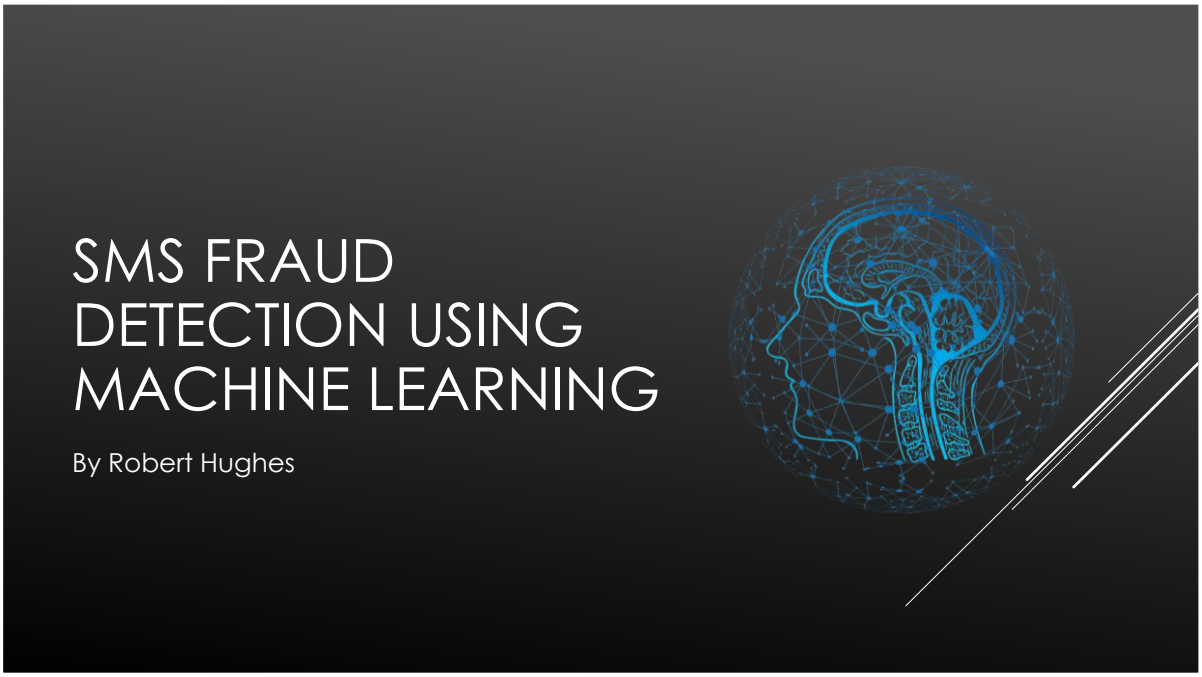
Chapter 9: References

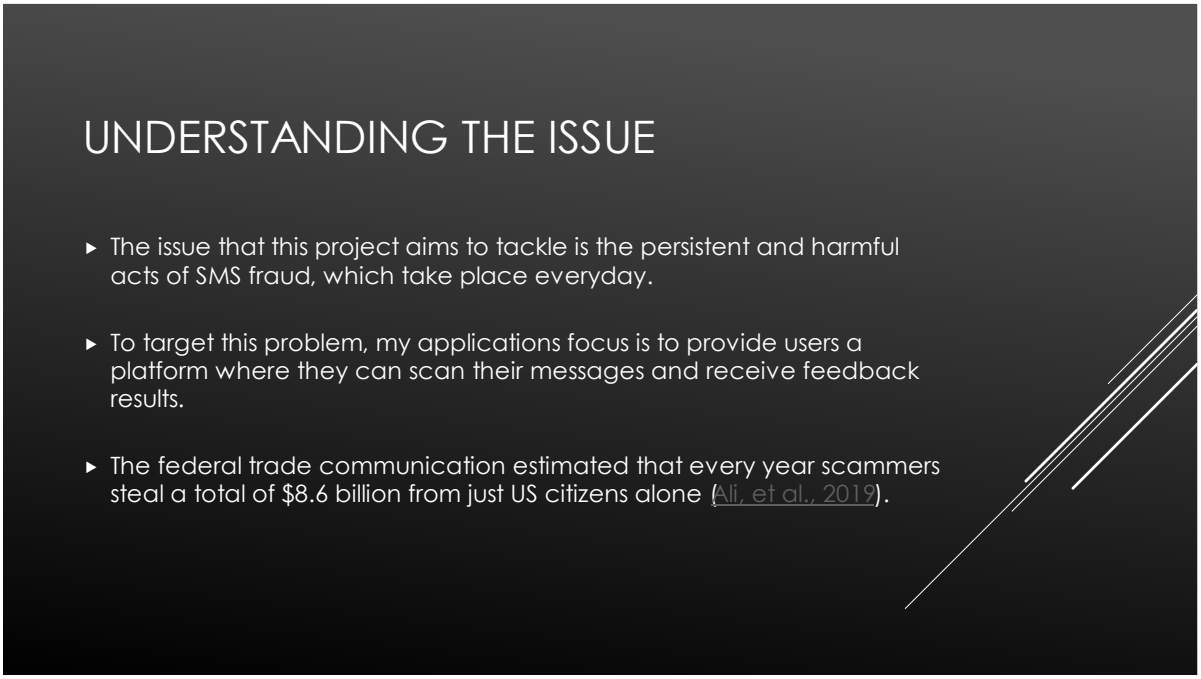
Agrawal, N., Bajpai, A., Dubey, K. & Patro, B., 2023. *An Effective Approach to Classify Fraud SMS Using Hybrid Machine Learning Models.* [Online]
Available at: https://ieeexplore.ieee.org/document/10126300
[Accessed 28 February 2024].
Akande, N. O., Gbenle, O. & Akiboy, C. O., 2023. *SMSPROTECT: An automatic smishing detection mobile application.* [Online]
Available at: https://www.sciencedirect.com/science/article/pii/S2405959522000868
[Accessed 29 February 2024].
Apple Inc, 2023. *Human Interface Guidelines: Typography.* [Online]
Available at: https://developer.apple.com/design/human-interface-guidelines/typography
[Accessed 2 May 2024].
Ghosh, A. & Senthilrajan, A., 2022. *A Modified Naïve Bayes Classifier for Detecting Spam E-mails based on Feature Selection.* [Online]
Available at: https://ieeexplore.ieee.org/document/9788340
[Accessed 12 March 2024].
Kavcic, A., 2006. *Software Accessibility: Recommendations and Guidelines.* [Online]
Available at: https://ieeexplore.ieee.org/document/1630123
[Accessed 6 May 2024].
Kumar, G. & Bhatia, P. K., 2012. *Impact of Agile Methodology on Software Development Process.* [Online]
Available at: https://www.researchgate.net/profile/Gaurav-Kumar-175/publication/255707851_Impact_of_Agile_Methodology_on_Software_Development_Process/links/00b49520489442e12d000000/Impact-of-Agile-Methodology-on-Software-Development-Process.pdf
[Accessed 5 May 2024].
Ma Ma, T., YAMAMORI, K. & Thida, A., 2020. *A Comparative Approach to Naïve Bayes Classifier and Support Vector Machine for Email Spam Classification.* [Online]
Available at: https://ieeexplore.ieee.org/document/9291921
[Accessed 12 March 2024].
Microsoft, n.d.. *FastForestBinaryTrainer Class.* [Online]
Available at: https://learn.microsoft.com/en-us/dotnet/api/microsoft.ml.trainers.fasttree.fastforestbinarytrainer?view=ml-dotnet
[Accessed 17 May 2024].
Mishra, P. et al., 2020. *New data preprocessing trends based on ensemble of multiple preprocessing techniques.* [Online]
Available at: https://doi.org/10.1016/j.trac.2020.116045
[Accessed 17 May 2024].
Portio Research Limited, 2016. *Mobile Messaging Futures 2012-2016.* [Online]
Available at: https://mobilemarketingmmk.files.wordpress.com/2013/03/mobile-messaging-futures-2012-2016-portio-research-ltd-full-report.pdf
[Accessed 13 April 2024].
Salman, M., Ikram, M. & Ali Kaafar, M., 2024. *Investigating Evasive Techniques in SMS Spam Filtering: A Comparative Analysis of Machine Learning Models.* [Online]
Available at: https://ieeexplore.ieee.org/document/10431737
[Accessed 28 February 2024].
Singh, B. & Gautam, S., 2016. *The Impact of Software Development Process on Software Quality: A Review.* [Online]
Available at: https://ieeexplore.ieee.org/document/8082729
[Accessed 11 April 2024].
Singh, G., Kumar, B., Gaur, L. & Tyagi, A., 2019. *Comparison between Multinomial and Bernoulli*

Naïve Bayes for Text Classification. [Online]
Available at: https://ieeexplore.ieee.org/document/8776800
[Accessed 1 April 2024].
Singh, T., Kumar, T. A. & Shambharkar, P. G., 2022. *Enhancing Spam Detection on SMS performance using several Machine Learning Classification Models.* [Online]
Available at: https://ieeexplore.ieee.org/abstract/document/9777157
[Accessed 3 March 2024].
Sneha, K. & Malle, G. M., 2018. *Research on software testing techniques and software automation testing tools.* [Online]
Available at: https://ieeexplore.ieee.org/document/8389562
[Accessed 1 April 2024].
V P, S., V, V., R, K. & T, T. T., 2023. *Performance Comparison of Machine Learning Algorithms in Short Message Service Spam Classification.* [Online]
Available at: https://ieeexplore.ieee.org/document/10199265
[Accessed 14 April 2024].
Yates, D. & Islam, M. Z., 2021. *FastForest: Increasing random forest processing speed while maintaining accuracy.* [Online]
Available at: https://doi.org/10.1016/j.ins.2020.12.067
[Accessed 1 May 2024].
Zhang, J. M., Harman, M., Ma, L. & Liu, Y., 2020. *Machine Learning Testing: Survey, Landscapes and Horizons.* [Online]
Available at: https://ieeexplore.ieee.org/abstract/document/9000651
[Accessed 3 April 2024].

Appendix A

Creative Piece Showcase Slides,





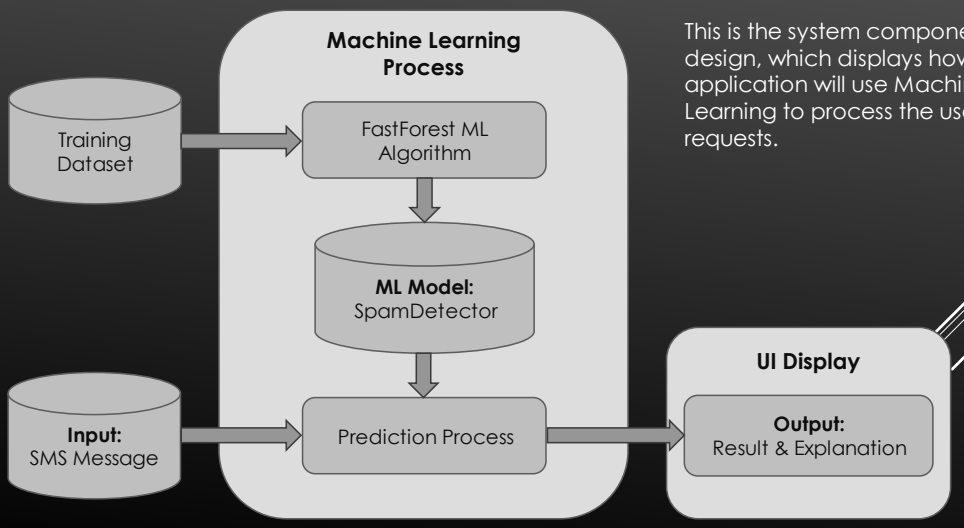
SOLUTION & AIMS

To solve this problem, this app aimed to provide users a high-quality platform to submit and scan their messages.

AIMS

- Create an efficient and high performing ML Model
- Train and test the model using a large dataset of spam/ham messages.
- Display quick, instant, and accurate predictions to the user.
- Provide an easy to understand and use interface.
- Conduct a thorough evaluation and analysis of the final application

SYSTEM COMPONENT DESIGN



This is the system component design, which displays how the application will use Machine Learning to process the user requests.

PROTOTYPE

The images to the right display the initial SMS detection prototype that was created in python.

This prototype worked successfully; however, I aimed to create a multi-platform tool and provide a better user interface and user experience.





TRAINING THE MODEL

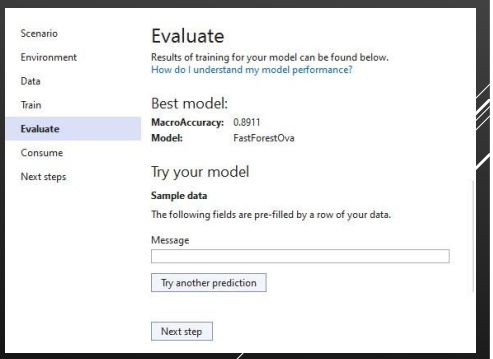
To train the model I used a SMS spam/ham dataset provided by Kaggle, which contained over 5,000 labelled messages.

Half of the dataset would be used for training the model and the other half would help evaluate and test its performance

	PRIVATE! Your 2003 Account Statement for shows 800 un-redeemed S. I. M. points. Call 08719899230 Identifier
1	Code: 41685 Expires 07/11/04
	Never blame a day in ur life. Good days give u happiness. Bad days give u experience. Both are essential in life! All are
0	Gods blessings! good morning.:
0	Pls confirm the time to collect the cheque.
	As a Registered Subscriber yr draw 4 a å£100 gift voucher will b entered on receipt of a correct ans. When are the next
1	olympics. Txt ans to 80062
	URGENT! Your Mobile number has been awarded with a å£2000 prize GUARANTEED. Call 09061790121 from land
1	line. Claim 3030. Valid 12hrs only 150ppm

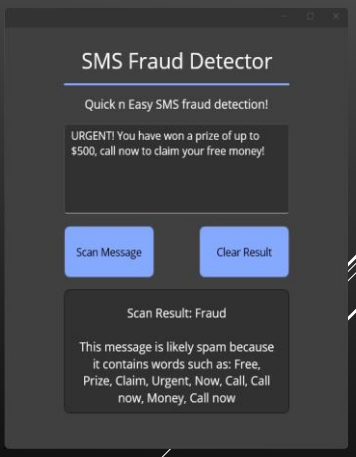
EVALUATION

- When evaluating the model, it achieved an accuracy score of 89%.
- The rigorous testing found that there were some false negatives / positives that with further development would need to be resolved.



FINAL APPLICATION

- My final product provided a simplistic user interface, with easy to use and understand control buttons.
- The finished app was able to receive, analyse and provide the user with accurate results.
- With this the app was able to provide an explanation on how the language used led to this result .



Appendix B

Visual Studio – User Interface Code (Main Page),

```
<StackLayout Grid.Row="1"
             Grid.ColumnSpan="2"
             HorizontalOptions="Center">
    <Label Text="SMS Fraud Detector"
           SemanticProperties.HeadingLevel="Level1"
           FontSize="32"
           HorizontalOptions="CenterAndExpand"
           Margin="0,0,0,12"/>
    <Rectangle BackgroundColor="#84A9FF"
               Margin="0,0,0,0"
               HeightRequest="3"
               MinimumWidthRequest="350"
               MaximumWidthRequest="350" />
    <Label Text="Quick n Easy SMS fraud detection!"
           SemanticProperties.HeadingLevel="Level2"
           FontSize="18"
           Margin="0,14,0,0"
           HorizontalOptions="Center" />
</StackLayout>

<ScrollView Grid.Row="2"
            Grid.ColumnSpan="2">
    <VerticalStackLayout

        Spacing="15"
        Padding="30,0"
        VerticalOptions="Center"
        HorizontalOptions="Center">

        <Editor x:Name="UserInput"
                Placeholder="Enter your message here. . ."
                Text="{Binding ResultText}"
                TextColor="White"
                HeightRequest="130"
                BackgroundColor="#313131"
                WidthRequest="350"
                Grid.ColumnSpan="2"
                FontSize="16" />
    </VerticalStackLayout>
</ScrollView>

<Frame Grid.Column="0"
       Grid.Row="3"
       Margin="0,0,35,0"
       HorizontalOptions="End"
       BorderColor="Transparent"
       BackgroundColor="Transparent"
       HasShadow="true"
       Padding="0"
       HeightRequest="75">
    <Button Command="{Binding ScanCommand}"
            WidthRequest="140"
            Text="Scan Message"
            HeightRequest="75"
            FontSize="15.5"
            BackgroundColor="#84A9FF"
            TextColor="Black"/>
</Frame>
```

```
87          <Frame Grid.Column="1"
88                 Grid.Row="3"
89                 Margin="35,0,0,0"
90                 HorizontalOptions="Start"
91                 HasShadow="true"
92                 Padding="0"
93                 HeightRequest="75"
94                 BorderColor="Transparent"
95                 BackgroundColor="Transparent">
96
97              <Button Command="{Binding ClearCommand}"
98                      WidthRequest="140"
99                      Text="Clear Result"
100                     FontSize="15.5"
101                     HeightRequest="75"
102                     BackgroundColor="#84A9FF"
103                     TextColor="Black"/>
104         </Frame>
105         <VerticalStackLayout Grid.Row="4" Grid.ColumnSpan="4">
106             <Frame WidthRequest="350"
107                    Margin="20,0,20,0"
108                   BackgroundColor="#313131"
109                   HeightRequest="180">
110                 <StackLayout>
111
112                     <Label Text="{Binding PredictionResult, StringFormat='Scan Result: {0}'}"
113                            FontSize="18"
114                            HorizontalTextAlignment="Center"
115                            TextColor="White"/>
116
117             </StackLayout>
118         </Frame>
119         </VerticalStackLayout>
120
121         <ActivityIndicator IsRunning="{Binding IsBusy}"
122                            IsVisible="{Binding IsBusy}"
123                            HeightRequest="40"
124                            WidthRequest="40"
125                            Grid.Row="4"
126                            Grid.ColumnSpan="4"
127                            VerticalOptions="CenterAndExpand"
128                            HorizontalOptions="CenterAndExpand" />
129     </Grid>
130 </vl:MainPageBase>
131
```

Appendix C

Prediction Code (PredictionViewModel.cs Page),

```
namespace SynopticApp.UI.VM
{
    1 reference
    public class PredictionViewModel :BaseViewModel
    {
        // defining proeprties for the user input - SMS message
        private string _resultText;
        2 references
        public string ResultText
        {
            get => _resultText;
            set => SetProperty(ref _resultText, value);

        }

        // defining the proprty to hold the prediction result - Spam or Safe
        private string _predictionResult;
        2 references
        public string PredictionResult
        {
            get => _predictionResult;
            set => SetProperty(ref _predictionResult, value);
        }

        // Creating commands for scanning the message, and clearing the output result area
        1 reference
        public ICommand ScanCommand { get; }
        1 reference
        public ICommand ClearCommand { get; }

        0 references
        public PredictionViewModel()
        {
            ScanCommand = new Command(async () => await Scan());
            ClearCommand = new Command(async () => await ClearResult());
        }
        //-------
```

```
        // Scan Message task that will give the user entered message to the ML Model
        1 reference
        public async Task Scan()
        {
            // Setting up relevant variables and setting the busy state to true.
            string ResultLabel;
            string ResultExplanation;
            IsBusy = true;

            PredictionResult = await Task.Run(string () =>
            {
                // Here the user ML 'message' is set to the user entered message,
                // and given to the SpamDetector Model as a new input for it to predict.
                var sampleData = new SpamDetector.ModelInput()
                {
                    Message = ResultText
                };

                // This if statement then determines the user output from the boolean results.
                // If the predicted label is 1, set the result label to 'Fraud' including its explanation
                if (SpamDetector.Predict(sampleData).PredictedLabel == 1)
                {
                    ResultLabel = "Fraud\n\n";

                    // Defined in Synoptic/ML/Explanation.cs
                    ResultExplanation = Explanation.GenerateExplanation(ResultText);

                    // simply connecting them to one output to return
                    return ResultLabel + ResultExplanation;
                }
                // if result is not 1, then it is safe.
                else
                {
                    return "Safe \n \n While this messages seems safe, never share personal" +
                    " information, or transfer money to unknown users.";
                }


            });
            // Process finished, busy = false and the user can now scan another message.
            IsBusy = false;
        }


        // Simple clear button to clear the Output result box.
        1 reference
        private Task ClearResult()
        {
            PredictionResult = null;
            return Task.CompletedTask;
        }
    }
}
```

Appendix D

ML Model Consumption (SpamDetector.consumption.cs Page),

```
namespace SynopticApp
{
    4 references
    public partial class SpamDetector
    {
        /// <summary>
        /// model input class for SpamDetector.
        /// </summary>
        model input class

        /// <summary>
        /// model output class for SpamDetector.
        /// </summary>
        model output class


        public static readonly Lazy<PredictionEngine<ModelInput, ModelOutput>> PredictEngine = new
            Lazy<PredictionEngine<ModelInput, ModelOutput>>(() => CreatePredictEngine().Result, true);

        // This page was auto generated by the ML Model Builder,
        // However this was altered to use the model stored in:
        // SynopticApp//Resources//Raw//SpamDetector.mlnet
        1 reference
        private static async Task<PredictionEngine<ModelInput, ModelOutput>> CreatePredictEngine()
        {
            var mlContext = new MLContext();
            using var stream = await FileSystem.OpenAppPackageFileAsync("SpamDetector.mlnet");
            ITransformer mlModel = mlContext.Model.Load(stream, out var _);
            return mlContext.Model.CreatePredictionEngine<ModelInput, ModelOutput>(mlModel);
        }


        /// <summary>
        /// Use this method to predict scores for all possible labels.
        /// </summary>
        /// <param name="input">model input.</param>
        /// <returns><seealso cref=" ModelOutput"/></returns>
        0 references
        public static IOrderedEnumerable<KeyValuePair<string, float>> PredictAllLabels(ModelInput input)
        {
            var predEngine = PredictEngine.Value;
            var result = predEngine.Predict(input);
            return GetSortedScoresWithLabels(result);
        }
```

```
        /// <summary>
        /// Map the unlabeled result score array to the predicted label names.
        /// </summary>
        /// <param name="result">Prediction to get the labeled scores from.</param>
        /// <returns>Ordered list of label and score.</returns>
        /// <exception cref="Exception"></exception>
        1 reference
        public static IOrderedEnumerable<KeyValuePair<string, float>> GetSortedScoresWithLabels(ModelOutput result)
        {
            var unlabeledScores = result.Score;
            var labelNames = GetLabels(result);

            Dictionary<string, float> labledScores = new Dictionary<string, float>();
            for (int i = 0; i < labelNames.Count(); i++)
            {
                // Map the names to the predicted result score array
                var labelName = labelNames.ElementAt(i);
                labledScores.Add(labelName.ToString(), unlabeledScores[i]);
            }

            return labledScores.OrderByDescending(c => c.Value);
        }

        /// <summary>
        /// Get the ordered label names.
        /// </summary>
        /// <param name="result">Predicted result to get the labels from.</param>
        /// <returns>List of labels.</returns>
        /// <exception cref="Exception"></exception>
        1 reference
        private static IEnumerable<string> GetLabels(ModelOutput result)
        {
            var schema = PredictEngine.Value.OutputSchema;

            var labelColumn = schema.GetColumnOrNull("Label");
            if (labelColumn == null)
            {
                throw new Exception("Label column not found. Make sure the name searched for matches the name in the schema.");
            }

            // Key values contains an ordered array of the possible labels. This allows us to map the results to the correct label value.
            var keyNames = new VBuffer<float>();
            labelColumn.Value.GetKeyValues(ref keyNames);
            return keyNames.DenseValues().Select(x => x.ToString());
        }

        /// <summary>
        /// Use this method to predict on <see cref="ModelInput"/>.
        /// </summary>
        /// <param name="input">model input.</param>
        /// <returns><seealso cref=" ModelOutput"/></returns>

        public static ModelOutput Predict(ModelInput input)
        {
            var predEngine = PredictEngine.Value;
            return predEngine.Predict(input);
        }
    }
}
```